

Análise da Modelagem Computacional do Problema Clássico de Otimização Combinatória "MOCHILA"

Gustavo Ruiz de Queiroz¹, Ricardo Rodrigues Affonso¹

¹Departamento de Informática – Universidade Tecnológica Federal do Paraná (UTFPR)

gusque@alunos.utfpr.edu.br, ricardoaffonso@alunos.utfpr.edu.br

Resumo. Este artigo apresenta uma análise comparativa de duas metaheurísticas aplicadas ao Problema da Mochila 0-1: Têmpera Simulada e Algoritmos Genéticos. O Problema da Mochila é um desafio clássico de otimização combinatória classificado como NP-difícil. O estudo foca na modelagem computacional e em como diferentes estratégias de exploração do espaço de busca influenciam a qualidade das soluções e o tempo de convergência. Foram realizados testes experimentais variando o número de itens e a capacidade da mochila. Os resultados demonstram que, a Têmpera Simulada apresenta ser a abordagem mais eficaz nas instâncias iniciadas em relação ao Algoritmo Genético, tanto em tempo de execução como em conseguir achar uma melhor solução para o problema.

Abstract. This article presents a comparative analysis of two metaheuristics applied to the 0-1 Knapsack Problem: Simulated Annealing and Genetic Algorithms. The Knapsack Problem is a classic combinatorial optimization challenge classified as NP-hard. The study focuses on computational modeling and on how different search space exploration strategies influence the quality of the solutions and the convergence time. Experimental tests were carried out by varying the number of items and the knapsack capacity. The results demonstrate that Simulated Annealing is the most effective approach for the evaluated instances when compared to the Genetic Algorithm, both in terms of execution time and solution quality.

Palavras-Chave: Problema da Mochila, Análise de Algoritmos, Modelagem Computacional, Têmpera Simulada, Algoritmo Genético.

1. Introdução

O Problema da Mochila (*Knapsack Problem*) constitui um dos problemas clássicos da área de otimização combinatória e da Ciência da Computação [Kellerer et al. 2004], sendo amplamente estudado em Pesquisa Operacional, Inteligência Artificial e Teoria da Complexidade. Seu objetivo consiste em selecionar, dentre um conjunto finito de itens, aqueles que maximizem o valor total transportado sem exceder uma capacidade máxima predefinida. Na formulação clássica 0-1, cada item pode ser selecionado apenas uma única vez, caracterizando um espaço de busca discreto e exponencial. Formalmente, dado um conjunto de n itens, em que cada item i possui peso w_i e valor v_i , deseja-se encontrar um vetor binário $x = (x_1, x_2, \dots, x_n)$, onde $x_i \in \{0, 1\}$, que maximize a seguinte função objetivo:

$$\max \sum_{i=1}^n v_i x_i \quad (1)$$

sujeito à restrição:

$$\sum_{i=1}^n w_i x_i \leq W \quad (2)$$

em que W representa a capacidade máxima da mochila.

Por se tratar de um problema pertencente à classe dos problemas NP-difíceis [Kellerer et al. 2004], sua resolução exata torna-se computacionalmente inviável para instâncias de grande porte, o que motiva a utilização de metaheurísticas capazes de produzir soluções aproximadas de alta qualidade em tempo computacional reduzido [Zhou et al. 2023].

Nesse contexto, destacam-se abordagens como a técnica de Têmpera Simulada (*Simulated Annealing*) e os Algoritmos Genéticos [Russell and Norvig 2013, Zhou et al. 2023], amplamente empregados na solução de problemas combinatórios complexos.

A modelagem computacional adotada em ambos os enfoques parte da representação da solução como um vetor binário, no qual cada posição corresponde a um item do problema. O valor 1 indica a inclusão do item na mochila, enquanto o valor 0 representa sua exclusão. Essa modelagem reflete diretamente a formulação matemática do problema, pois o vetor de decisão é o elemento central tanto na avaliação da função objetivo quanto na verificação das restrições de capacidade.

No método de Têmpera Simulada, essa modelagem se manifesta na estrutura de vizinhança da solução. A partir de uma solução inicial, novas soluções candidatas são geradas por pequenas perturbações no vetor binário, como a inversão de um ou mais bits. Dessa forma, a modelagem é refletida diretamente no mecanismo de busca local, em que a alteração dos bits representa a exploração do espaço de soluções. A função de custo utilizada pelo algoritmo corresponde à função objetivo do problema, podendo incluir penalizações para soluções inviáveis que ultrapassem a capacidade da mochila.

Já no enfoque de Algoritmo Genético, a modelagem binária é refletida diretamente na representação do cromossomo. Cada indivíduo da população corresponde a uma solução possível do problema, codificada como uma sequência binária. Os operadores genéticos, como *crossover* e mutação, atuam diretamente sobre essa estrutura, combinando e modificando cromossomos para gerar novas soluções. A busca local, por sua vez, é incorporada como um mecanismo de refinamento aplicado aos indivíduos gerados, promovendo melhorias pontuais por meio da exploração de sua vizinhança imediata.

Diante disso, o presente artigo tem como objetivo realizar uma análise comparativa da modelagem computacional do Problema da Mochila sob os enfoques da Têmpera Simulada e do Algoritmo Genético, investigando como a representação formal do problema influencia a implementação, o comportamento e a qualidade das soluções obtidas por cada abordagem.

2. Metodologia

Para a avaliação dos algoritmos, foram geradas instâncias do Problema da Mochila com 20 e 100 itens, variando também as capacidades máximas. Os pesos e valores dos itens foram definidos de forma pseudoaleatória, utilizando a mesma semente de geração para ambos os algoritmos, garantindo a comparabilidade dos resultados.

A função de avaliação da qualidade da solução foi definida como a soma dos valores dos itens selecionados, desde que a soma dos respectivos pesos não ultrapassasse a capacidade máxima da mochila.

2.1. Implementação da Têmpera Simulada

O algoritmo de Têmpera Simulada foi configurado com um esquema de resfriamento geométrico, definido por $T_{k+1} = \alpha T_k$, onde $\alpha = 0.98$. Essa escolha se deu por que o resfriamento geométrico é a implementação mais comum e eficiente desse tipo de função.

A temperatura inicial foi definida de modo a permitir uma alta probabilidade de aceitar soluções piores no início do processo, evitando ótimos locais. A função de vizinhança opera através do *bit-flip* (inversão de um bit aleatório, técnica comum em problemas de representação binária [Zhou et al. 2023]).

Foram utilizados os seguintes parâmetros:

- **Temperatura inicial:** 1000 (converge a 100% de aceitação de soluções piores no começo);
- **Temperatura final:** 0.01 (implica em aproximadamente 0% de tomar uma decisão ruim no final);
- **Taxa de resfriamento:** 0.98 (valor definido para promover redução gradual da temperatura, mas sem comprometer a busca pelo ótimo global);
- **Iterações por temperatura:** 100 (iteraões suficientes para explorar o espaço de busca).

Sua implementação foi baseada no livro Inteligência Artificial [Russell and Norvig 2013], resultando no pseudocódigo do algoritmo 1, que pode ser encontrado no Apêndice B.

2.2. Implementação do Algoritmo Genético

A quantidade de gerações foi definida de forma a manter uma ordem de grandeza semelhante ao número total de iterações executadas pela Têmpera Simulada, permitindo uma comparação experimental mais equilibrada entre as abordagens. Dessa forma, foi utilizada uma população de 100 indivíduos (gerados aleatoriamente no começo), onde o critério de parada do algoritmo foi definido pelo número máximo de 570 gerações. Assim, os parâmetros implementados foram:

- **Seleção:** Torneio (escolhe 3 genes aleatoriamente e pega o melhor);
- **Crossover:** Cruzamento com taxa de 90%;
- **Mutação:** Taxa de 5% por gene;
- **Elitismo:** Salva o melhor gene de cada geração (não perder a melhor solução);

A seleção é realizada por torneio, método amplamente empregado em algoritmos genéticos por equilibrar pressão seletiva e diversidade populacional [Russell and Norvig 2013]. As taxas de crossover e mutação foram definidas conforme práticas recorrentes na literatura de algoritmos evolutivos [Zhou et al. 2023]. Tais processos podem ser observados no algoritmo 2, que está localizado no Apêndice C.

3. Resultados

As implementações dos algoritmos foram disponibilizadas em um repositório público no GitHub. O link para acesso ao código-fonte encontra-se no Apêndice A.

Os experimentos foram executados em ambiente local utilizando Python 3.12.3, em máquina com processador i5 11th Gen 2.4GHz e 16 GB de memória RAM.

3.1. Resultados da Têmpera Simulada

Com a parametrização definida, o algoritmo executa aproximadamente 57.000 iterações. Conforme ilustrado nas Figuras D.2 e D.1, para uma instância de entrada contendo 20 itens, ele converge para a melhor solução encontrada em 0,23 segundos aproximadamente.

O algoritmo apresentou comportamento estável mesmo em problemas maiores. A figura D.4 mostra a eficácia da Têmpera Simulada ao encontrar soluções ótimas para 100 itens, no qual o tempo de execução para convergir para a solução de maior qualidade foi de aproximadamente 0,59 segundos.

Em relação à solução final, na simulação de 100 itens, o valor total gerado foi de 4564, e a capacidade ocupada apresentou 526/532, de acordo com o resultado da figura D.3.

3.2. Resultados do Algoritmo Genético

Quanto mais itens eram utilizados para as simulações, o Algoritmo Genético apresentou um tempo de execução proporcionalmente maior e uma variância mais acentuada nas soluções finais, isso quando comparado com os resultados do Algoritmo da Têmpera Simulada. Na simulação de 100 itens, o tempo de execução foi aproximadamente 2,42 segundos, conforme o gráfico de Gerações x Tempo de processamento da figura D.8.

Em relação às soluções finais obtidas, ainda na simulação de 100 itens, o Algoritmo Genético apresentou um valor total gerado de 4387, enquanto a capacidade total de 522/532, obtido esses resultados na geração 560, conforme a figura D.7.

Embora as Figuras D.6 e D.5 demonstrem que a melhor solução encontrada foi alcançada para a instância de 20 itens, as Figuras D.8 e D.7 revelam que, para 100 itens, os valores encontrados eram significativamente diferentes daqueles obtidos pelo método anterior.

3.3. Comparação dos Algoritmos

Ao analisar instâncias com menor número de itens, ambos os algoritmos convergiram para o mesmo resultado. Contudo, devido ao menor custo computacional por iteração, a Têmpera Simulada obteve um tempo de execução aproximadamente 3,5 vezes inferior quando comparado ao Algoritmo Genético.

Em problemas mais complexos (100 itens), a Têmpera Simulada demonstrou clara superioridade. A alta taxa de exploração nos estágios iniciais de temperatura garantiu a convergência para a melhor solução encontrada em um tempo menor. Observou-se um tempo de execução de aproximadamente 410% mais lento para o Algoritmo Genético, conforme as figuras D.4 e D.8.

A Figura D.9 apresenta uma análise comparativa baseada em 20 execuções independentes de cada algoritmo, demonstrando graficamente a vantagem em tempo e qualidade da Têmpera Simulada sobre o Algoritmo Genético.

Em termos de qualidade da solução, a Têmpera Simulada apresentou valor final de 4564, superando o Algoritmo Genético, que atingiu 4387, diferença de aproximadamente 4,0%. A taxa de ocupação da mochila também favoreceu a Têmpera Simulada, com utilização de 98,8% da capacidade total, frente aos 98,1% observados no Algoritmo Genético.

4. Conclusão

Os resultados indicam que a representação binária adotada em ambos os algoritmos mostrou-se adequada ao problema, porém a forma como cada abordagem explora esse espaço revelou diferenças significativas de desempenho.

A Têmpera Simulada demonstrou ser a abordagem mais eficaz nas instâncias testadas, tanto em qualidade de solução quanto em tempo de execução. O esquema de resfriamento geométrico com $\alpha = 0,98$ permitiu uma transição gradual entre exploração e exploração, evitando mínimos locais e convergindo para soluções de alta qualidade.

O Algoritmo Genético, por sua vez, apresentou desempenho inferior, especialmente para 100 itens, devido à alta variância introduzida pelos operadores evolutivos, essa aleatoriedade presente (seleção por torneio, crossover e mutação), gerou muitas variações nas soluções a cada execução. Vale destacar que a incorporação efetiva de busca por vizinhança ao AG, poderia reduzir essa variância e melhorar consideravelmente os resultados. Contudo, é possível supor que o Algoritmo Genético possa apresentar vantagens em espaços de busca com tamanhos muito superiores, visto que sua tendência é explorar de maneira mais ampla.

Em síntese, os resultados reforçam que a escolha da metaheurística deve ser orientada pelas características específicas do problema, sendo a análise experimental criteriosa um passo indispensável nesse processo.

5. Papel de cada membro

O desenvolvimento deste artigo contou com a participação conjunta dos membros Gustavo Ruiz de Queiroz e Ricardo Rodrigues Affonso, ambos dedicando horas significativas à elaboração do trabalho. As quantidades de horas dependidas por cada etapa deste artigo pode ser encontrada na tabela do apêndice E

Gustavo Ruiz de Queiroz concentrou sua atuação, principalmente, na organização da estrutura do artigo, na elaboração da introdução e na pesquisa de fontes bibliográficas, além de também contribuir para o desenvolvimento da metodologia e para a análise e elaboração dos resultados obtidos.

Por sua vez, Ricardo Rodrigues Affonso direcionou seu foco, de forma mais expressiva, à elaboração da metodologia e ao desenvolvimento do algoritmo em código, etapa fundamental para a obtenção de resultados concretos e visíveis referentes às duas modelagens aplicadas. Adicionalmente, sua participação foi relevante na análise comparativa entre os algoritmos desenvolvidos, bem como na formulação da conclusão acerca da modelagem mais adequada para o problema estudado.

Referências

- Kellerer, H., Pferschy, U., and Pisinger, D. (2004). *Knapsack Problems*. Springer.
- Russell, S. and Norvig, P. (2013). *Artificial Intelligence: A Modern Approach*. Pearson, Boston, 3 edition.
- Zhou, Y., Shi, Y., Wei, Y., Luo, Q., and Tang, Z. (2023). Nature-inspired algorithms for 0-1 knapsack problem: A survey. *Neurocomputing*, 554:126630.

A. Apêndice: Acesso ao algoritmo em código

Link para o Github:

```
https://github.com/RicardoAffonso0607/  
Sistemas-Inteligentes/tree/main/artigo1
```

B. Apêndice: Pseudocódigo do algoritmo de Têmpera Simulada

Algorithm 1. Têmpera Simulada para o Problema da Mochila

Require: Pesos P , Valores V , Capacidade C , $T_{inicial}$, T_{final} , Taxa de resfriamento α , Iterações por temperatura L

Ensure: Melhor estado da mochila S_{melhor} e seu valor total V_{melhor}

```
1:  $S_{atual} \leftarrow [0, 0, \dots, 0]$  ▷ Inicia com a mochila vazia
2:  $S_{melhor} \leftarrow S_{atual}$ 
3:  $V_{atual} \leftarrow \text{AvaliaMochila}(S_{atual}, P, V, C)$ 
4:  $V_{melhor} \leftarrow V_{atual}$ 
5:  $t \leftarrow 0$  ▷ Tempo / Contador de iterações globais
6: while Verdadeiro do
7:    $ciclo \leftarrow \lfloor t/L \rfloor$ 
8:    $T \leftarrow T_{inicial} \times \alpha^{ciclo}$  ▷ Calcula a temperatura atual
9:   if  $T \leq T_{final}$  then
10:    interrompe o laço ▷ Critério de parada atingido
11:   end if
12:    $S_{vizinho} \leftarrow \text{GeraVizinho}(S_{atual})$  ▷ Inverte um item aleatório
13:    $V_{vizinho} \leftarrow \text{AvaliaMochila}(S_{vizinho}, P, V, C)$ 
14:    $\Delta \leftarrow V_{vizinho} - V_{atual}$  ▷ Diferença de qualidade
15:   if  $\Delta > 0$  then ▷ Maximização: aceita se o vizinho for melhor
16:      $S_{atual} \leftarrow S_{vizinho}$ 
17:      $V_{atual} \leftarrow V_{vizinho}$ 
18:     if  $V_{atual} > V_{melhor}$  then
19:        $S_{melhor} \leftarrow S_{atual}$ 
20:        $V_{melhor} \leftarrow V_{atual}$ 
21:     end if
22:   else ▷ Se for pior, tenta aceitação probabilística
23:      $probabilidade \leftarrow \exp(\Delta/T)$ 
24:      $r \leftarrow \text{Valor aleatório uniforme em } [0, 1)$ 
25:     if  $r < probabilidade$  then
26:        $S_{atual} \leftarrow S_{vizinho}$ 
27:        $V_{atual} \leftarrow V_{vizinho}$ 
28:     end if
29:   end if
30:    $t \leftarrow t + 1$ 
31: end while
32: return  $S_{melhor}, V_{melhor}$ 
```

C. Apêndice: Pseudocódigo do Algoritmo Genético

Algorithm 2. Algoritmo Genético para o Problema da Mochila

Require: População Inicial P , Número de Gerações G , Itens I , Capacidade C , Taxa de Crossover α_c , Taxa de Mutação α_m

Ensure: Melhor indivíduo encontrado S_{melhor} e seu valor V_{melhor}

```
1:  $N \leftarrow \text{Tamanho}(P)$ 
2: for  $g \leftarrow 1 \dots G$  do
3:    $P_{nova} \leftarrow []$  ▷ Inicia a nova geração vazia
4:   ▷ Elitismo: preserva o indivíduo mais apto da geração atual
5:    $Melhor_{atual} \leftarrow \text{Indivíduo com maior Aptidão em } P$ 
6:   Adiciona  $Melhor_{atual}$  em  $P_{nova}$ 
7:   while  $\text{Tamanho}(P_{nova}) < N$  do
8:   ▷ Seleção de pais via Torneio
9:      $Pai_1 \leftarrow \text{SelecaoTorneio}(P, I, C)$ 
10:     $Pai_2 \leftarrow \text{SelecaoTorneio}(P, I, C)$ 
11:    ▷ Cruzamento e Mutação
12:     $Filho_1, Filho_2 \leftarrow \text{Crossover}(Pai_1, Pai_2, \alpha_c)$ 
13:     $Filho_1 \leftarrow \text{Mutacao}(Filho_1, \alpha_m)$ 
14:     $Filho_2 \leftarrow \text{Mutacao}(Filho_2, \alpha_m)$ 
15:    Adiciona  $Filho_1$  e  $Filho_2$  em  $P_{nova}$ 
16:   end while
17:    $P \leftarrow P_{nova}[1 \dots N]$  ▷ Garante o tamanho exato da população truncando sobras
18: end for
19:  $S_{melhor} \leftarrow \text{Indivíduo com maior Aptidão em } P$ 
20:  $V_{melhor} \leftarrow \text{CalculaFitness}(S_{melhor}, I, C)$ 
21: return  $S_{melhor}, V_{melhor}$ 
```

D. Apêndice: Figuras

D.1. Terminal da Têmpera Simulada com 20 itens

```
Tue 7 Apr - 09:47 ~/Desktop/university/sisInt/artigo1 origin main 1* 2
@ricardo python3 temp_sim.py
Iniciando Têmpera Simulada...

=====
RESULTADO FINAL
=====
Melhor combinação: [1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1]
Valor Total Gerado: 927
Capacidade Ocupada: 92 / 104
```

Figura 1. Terminal da Têmpera Simulada com 20 itens

D.2. Gráfico da Têmpera Simulada com 20 itens

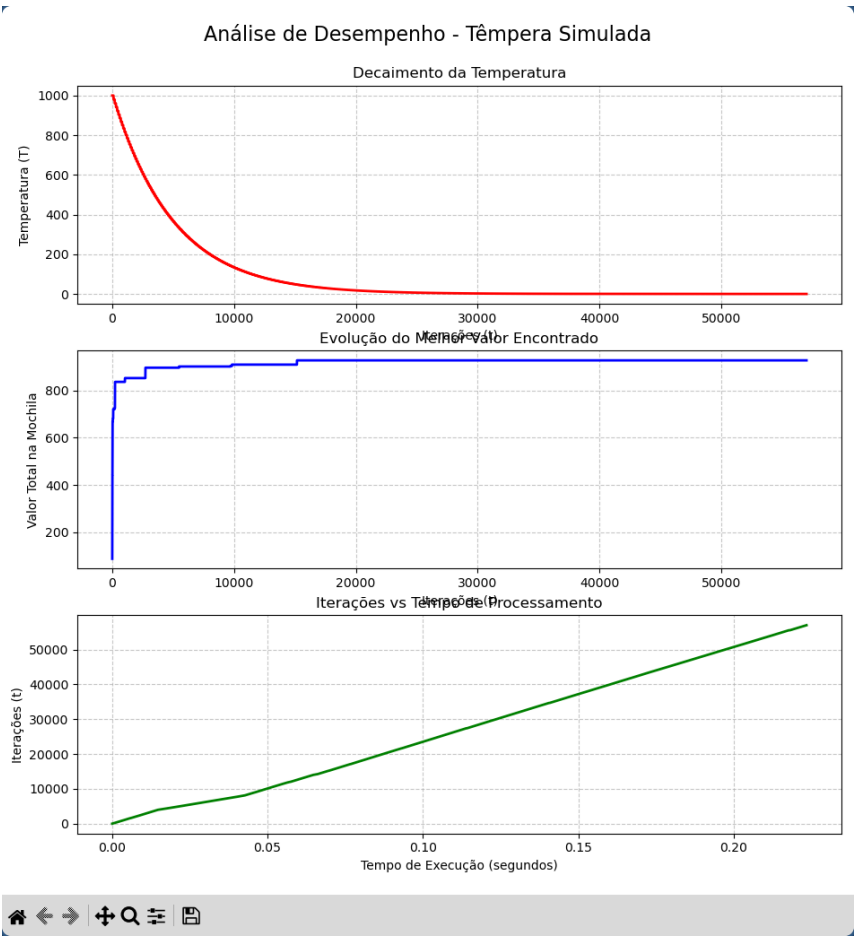


Figura 2. Gráfico da Têmpera Simulada com 20 itens

D.3. Terminal da Têmpera Simulada com 100 itens

```
Tue 7 Apr - 10:17 ~/Desktop/university/sisInt/artigo1 origin main 1*
@ricardo python3 temp_sim.py
Iniciando Têmpera Simulada...

=====
RESULTADO FINAL
=====
Melhor combinação: [0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1
, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1
, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1
, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1
, 1, 0]
Valor Total Gerado: 4564
Capacidade Ocupada: 526 / 532
```

Figura 3. Terminal da Têmpera Simulada com 100 itens

D.4. Gráfico da Têmpera Simulada com 100 itens

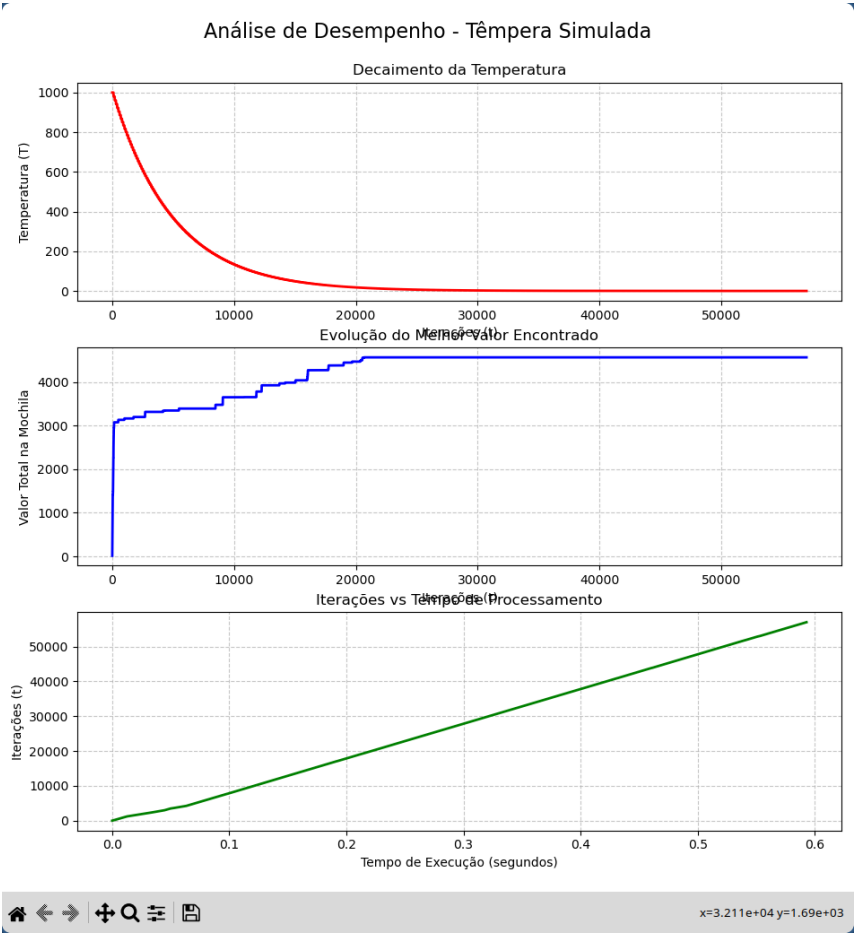


Figura 4. Gráfico da Têmpera Simulada com 100 itens

D.5. Terminal do Algoritmo Genético com 20 itens

```
=====
RESULTADO FINAL
=====
Melhor Cromossomo:  [1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1]
Valor Total Gerado: 927
Capacidade Ocupada: 92 / 104
```

Figura 5. Terminal do Algoritmo Genético com 20 itens

D.6. Gráfico do Algoritmo Genético com 20 itens

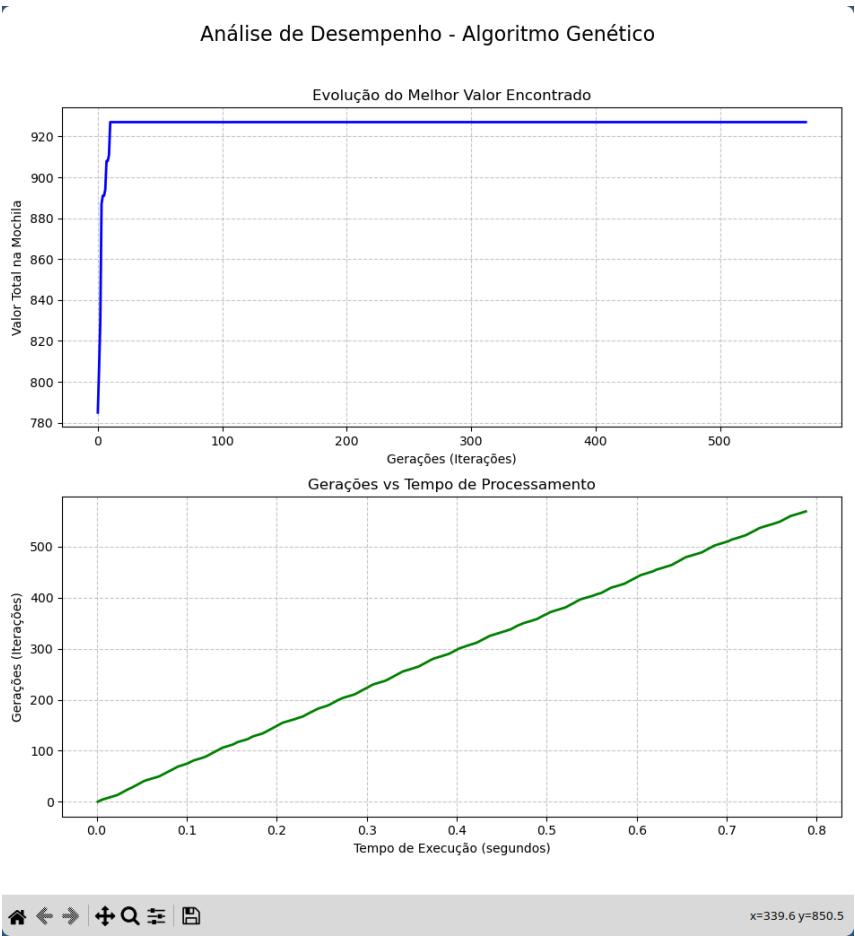


Figura 6. Gráfico do Algoritmo Genético com 20 itens

D.7. Terminal do Algoritmo Genético com 100 itens

```
Tue 7 Apr - 11:11 ~/Desktop/university/sisInt/artigo1 origin main 2
...
Geração 560 | Melhor Valor: 4387

=====
RESULTADO FINAL
=====
Melhor Cromossomo: [0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1,
0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1,
1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1,
1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1,
1, 1, 0]
Valor Total Gerado: 4387
Capacidade Ocupada: 522 / 532
```

Figura 7. Terminal do Algoritmo Genético com 100 itens

D.8. Gráfico do Algoritmo Genético com 100 itens

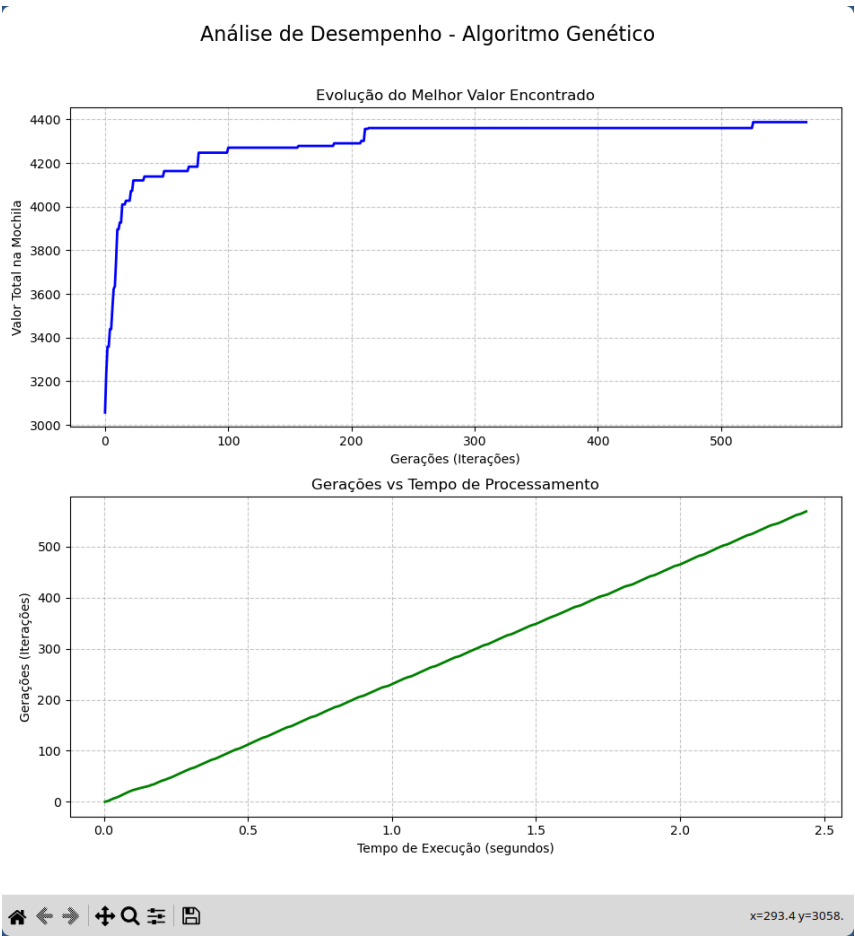


Figura 8. Gráfico do Algoritmo Genético com 100 itens

D.9. Gráfico da comparação dos dois algoritmos com 100 itens

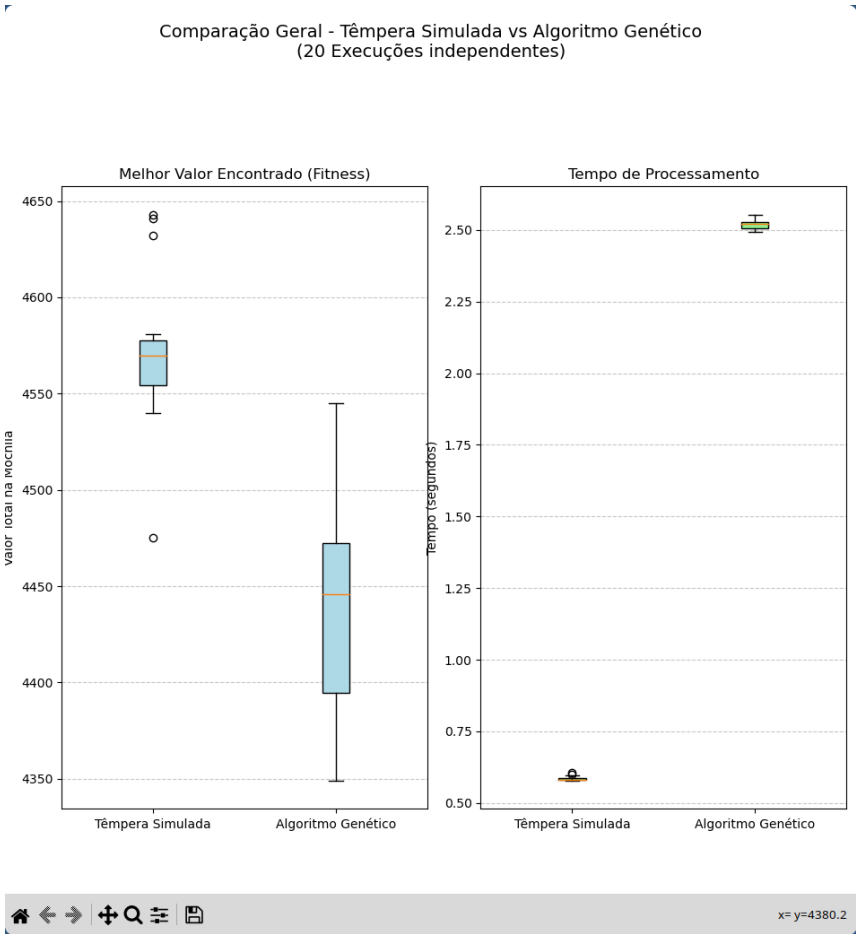


Figura 9. Gráfico da comparação dos dois algoritmos com 100 itens

E. Tabela de horas despendidas

Estimativa das horas despendidas por cada integrante	Gustavo Ruiz de Queiroz	Ricardo Rodrigues Affonso
Estudo do conteúdo e busca de fontes	8 horas	10 horas
Elaboração da introdução	8 horas	2 horas
Elaboração da metodologia	3 horas	4 horas
Elaboração dos resultados	5 horas	5 horas
Elaboração do algoritmo em código	–	7 horas
Elaboração da conclusão	4 horas	3 horas
Organização da estrutura do artigo	6 horas	3 horas
Revisão dos tópicos	5 horas	2 horas

Tabela 1. Estimativa das horas despendidas por cada integrante